Day 90: The Week 13 Build — a Tiny Character-Level GPT-Style Model on Shakespeare

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 7 | Complete study guide

Watch: The spelled-out intro to language modeling: building makemore (Andrej Karpathy) | Transformers, the tech behind LLMs, Deep Learning Chapter 5 (3Blue1Brown)

Script: `day90_tiny_gpt.py` · every number in this guide is copied by program from `day90_run_output.txt`, the real output of that script.

Week 13's six exercise parts and Week 12's Transformer block, put together into a tiny GPT-style model and trained on Shakespeare's plays. It writes plausible Shakespeare-shaped text, and its learned character table puts capitals next to small letters and '?' next to '!'. Measured honestly, it is still small: scored with enough context it needs 2.525 bits per character, a tie with a model that just counts what follows each four characters (2.554). The same model on BPE pieces does better (2.399), partly because its 64-token window covers about twice as much text.

0. Where today fits

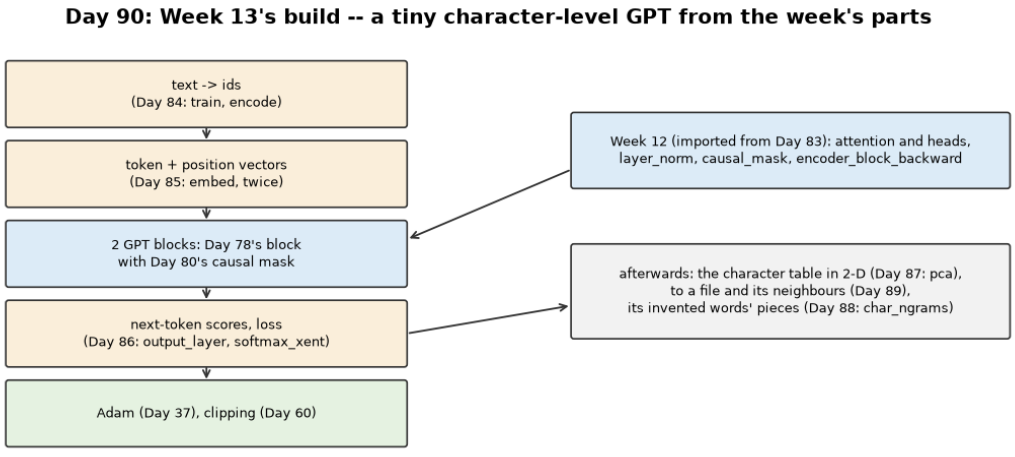
DEFINITION — start here, no prior knowledge assumed

A **language model** gives a probability for the next token given the ones before it. A **GPT-style** model is a stack of Transformer blocks whose attention may only look backwards (a **causal mask**): position t reads positions 0..t and predicts the token at t + 1. Trained on raw text with nothing labelled, it can then **write**: draw a token, append it, repeat. Quality is measured in **bits per character** on text it never trained on: how many yes/no questions, on average, it still needs to guess each next character.

REAL-WORLD SCENARIO

A phone keyboard's next-word suggestion, at the level of single letters, trained on one author.

SYSTEM DIAGRAM



Week 13's build: the week's parts, Week 12's block, one trained model.

The parts

From	Part	Where it goes
Day 84 exercise	merge, train, encode, decode	text to ids: characters (0 merges) or BPE
Day 85 exercise	embed, embed_backward	the token table and the position table
Day 86 exercise	output_layer, softmax_xent	next-token scores and the loss
Day 87 exercise	pca	a picture of the character table
Day 88 exercise	char_ngrams	are invented words made of real pieces?
Day 89 exercise	save_vectors, load_vectors, most_similar	the table to a file; its neighbours
Week 12 (Day 83)	attention, heads, layer_norm, causal_mask, encoder_block_backward	the GPT block
Day 37	Adam's update rule	training

The one new piece of model code: Day 78's encoder block with Day 80's causal mask in place of the padding mask. Day 78's backward pass works unchanged, because the mask only decides which attention weights are 0.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The build fits

DEFINITION — start here, no prior knowledge assumed

The whole model at tiny sizes (10 tokens, length 6, two blocks), every weight checked against a numerical gradient; then the causal mask, end to end: change the token at position 4 and see which scores move.

REAL-WORLD SCENARIO

Before the first concert, every instrument is tuned and every cable tested.

MATHEMATICS

$$score_{ij} = q_i \cdot k_j / \sqrt{d_k} \text{ for } j \le i, -\infty \text{ for } j > i; \quad weights = softmax \text{ over } j$$

IMPLEMENTATION

```
def gpt_block(x, P, h):
    m, n, _ = x.shape
    km = np.broadcast_to(w12.causal_mask(n), (m * h, n, n))
    Q, K, V = (w12.split(x @ P[w], h) for w in ("Wq", "Wk", "Wv"))
    Zh, A = w12.attention(Q, K, V, km)
    Z = w12.merge(Zh, h)
    h1, n1 = w12.layer_norm(x + Z @ P["Wo"], P["g1"], P["b1"])
    u = h1 @ P["W1"] + P["c1"]; r = np.maximum(u, 0)
    y, n2 = w12.layer_norm(h1 + r @ P["W2"] + P["c2"], P["g2"], P["b2"])
    return y, (x, Q, K, V, A, Z, h1, n1, u, r, n2)
```

ACTUAL OUTPUT

```
gradient check: 1218 entries, largest |numeric - analytic| = 1.85e-10, every entry passes: True
changing the token at position 4: largest change in the scores at positions 0-3 0.0e+00, at positions 4-5
    4.1e-01 (the past cannot see the future)
the lesson's model: 2 blocks, d = 64, 4 heads, FFN 256, context 64: 111937 weights for 65 characters
Actual output from this lesson's run.
```

What it means

All 1218 entries pass (1.85e-10): Week 12's backward pass and Week 13's parts fit. Changing a later token moves the later scores (4.1e-01) and leaves the earlier ones exactly unchanged (0.0e+00). The lesson's model has 111937 weights.

WHY CHOOSE THIS?

Only one line differs from Day 78's block; everything else is reused and checked.

WHY NOT THIS? (TRADEOFFS)

Post-norm blocks (Day 78); GPT-2 and later put the norm first (Day 79).

2. Tokens: characters or BPE pieces

DEFINITION — start here, no prior knowledge assumed

Day 84's tokenizer with 0 merges gives one token per character (65 different characters in Shakespeare). With 200 merges it gives BPE pieces; spaces and newlines are ordinary characters, so a piece can be 'e ' or ', '.

REAL-WORLD SCENARIO

Writing in letters or in common syllables.

ACTUAL OUTPUT

```
characters: vocabulary 65, held-out text 111,540 characters -> 111,540 tokens (1.00 characters per token);
    decodes back exactly: True; 0 s
BPE: vocabulary 265, held-out text 111,540 characters -> 60,149 tokens (1.85 characters per token); decodes
    back exactly: True; 74 s
characters: "\n !$&',-.3:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
first BPE merges: 'e ' | 'th' | 't ' | 's ' | 'd ' | ', ' | 'ou' | 'er' | 'in' | 'y '
Actual output from this lesson's run.
```

What it means

Both tokenizations are lossless. BPE needs 1.85 characters per token, so a window of 64 tokens covers almost twice as much text, at the cost of a vocabulary four times larger.

WHY CHOOSE THIS?

Characters: tiny vocabulary, no unknown words.

WHY NOT THIS? (TRADEOFFS)

Long sequences: 64 characters is a sentence, not a scene.

3. What to beat: counting models

DEFINITION — start here, no prior knowledge assumed

An n-gram model counts, in the training text, how often each character followed each context of n – 1 characters. Add-k smoothing adds k to every count, so unseen combinations keep a small probability.

REAL-WORLD SCENARIO

Guessing the next letter from the last four alone, using a huge table of what usually follows them.

MATHEMATICS

$$p(c \mid context) = (count(context, c) + k) / (count(context) + k\,V); \quad bits\ per\ character = mean\ of\ -\log_2 p$$

ACTUAL OUTPUT

```
context              bits/char      k  contexts seen
0 characters             4.829   0.01              1
1 characters             3.583    0.1             65
2 characters             2.951    0.1          1,380
3 characters             2.574    0.1         11,228
4 characters             2.554   0.01         48,539
Actual output from this lesson's run.
```

What it means

A uniform guess costs $\log_2 65 = 6.02$ bits. Letter frequencies alone give 4.829; four characters of context give 2.554, barely better than three (2.574) because most 4-character contexts are rare. Add-k is the simplest smoothing; better smoothing (back-off, interpolation) would do better, so this is a baseline, not the best counting model.

WHY CHOOSE THIS?

Instant, exact counts; a strong baseline at short context.

WHY NOT THIS? (TRADEOFFS)

Contexts beyond a few characters are almost never seen twice.

4. Training, and how much context it really uses

DEFINITION — start here, no prior knowledge assumed

Adam (Day 37) on random 64-character windows, 4,000 steps, two seeds; held-out bits per character every 500 steps on all 1,742 windows of the last 10%. Each held-out window starts with no context, so its first characters are hard; two more measurements correct for that.

REAL-WORLD SCENARIO

Testing a reader by opening the book at random: the first words on the page make less sense than the rest.

SPECIFICATION

Model	2 blocks, d = 64, 4 heads, FFN 256, context 64, learned position table, output layer × 0.1.
Training	Adam, learning rate 0.003, clipping at 1.0, batches of 32 windows, 4,000 steps, seeds 0 and 1.
Fair comparison	the loss at each position of the window, and bits per character when every scored character has at least 32 characters of context.

Why this test

A counting model always sees its full context; the GPT, measured on windows, does not. The position curve shows both how much context the model uses and what the fair comparison is.

IMPLEMENTATION

```
def position_bits(P, vx, vy):
    tot = np.zeros(vx.shape[1])
    for s in range(0, len(vx), 64):
        logp = log_softmax(gpt_forward(vx[s:s + 64], P)[0])
        tot -= np.take_along_axis(logp, vy[s:s + 64][..., None], axis=-1)[..., 0].sum(axis=0) /
            np.log(2)
    return tot / len(vx)
def context_bits(P, ids, vocab):
    h = T // 2
    starts = np.arange(0, len(ids) - T - 1, h)
    nats, chars = 0.0, 0
    for s in range(0, len(starts), 64):
        st = starts[s:s + 64]
        x = np.stack([ids[i:i + T] for i in st]); y = np.stack([ids[i + 1:i + T + 1] for i in st])
        logp = log_softmax(gpt_forward(x, P)[0][:, h:])
        nats -= np.take_along_axis(logp, y[:, h:, None], axis=-1).sum()
        chars += sum(len(vocab[i]) for i in y[:, h:].ravel())
    return nats / chars / np.log(2)
```

ACTUAL OUTPUT

```
  step  train s0  held-out s0  train s1  held-out s1
     0         -        6.018         -        6.036
   500     3.345        2.988     3.356        3.023
  1000     2.726        2.866     2.746        2.837
  1500     2.570        2.731     2.583        2.775
  2000     2.480        2.688     2.502        2.704
  2500     2.433        2.627     2.446        2.679
  3000     2.392        2.623     2.410        2.657
  3500     2.361        2.594     2.374        2.614
  4000     2.338        2.564     2.355        2.614
(317 ms per training step here, held-out checks not counted)
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
The held-out windows above start with no context. Seed 0's held-out bits per character by position in the
    window (the prediction at position p is made from p + 1 characters):
  position 0: 3.68, position 1: 3.15, position 2: 2.87, position 4: 2.69, position 8: 2.58, position 16: 2.50,
      position 32: 2.49, position 63: 2.59
every character scored with at least 32 characters of context: seed 0 2.525, seed 1 2.578
Actual output from this lesson's run.
```

What it means

Held-out bits per character fall from 6.018 to 2.564 and 2.614 in 4,000 steps and are still falling. The model clearly uses context: with 1 character it needs 3.68 bits, with 5 2.69, with 17 2.50. Per character of context it is less efficient than counting (with 5 characters the counting model needs 2.554), but it can use far longer contexts. Scored fairly, with at least 32 characters of context, it reaches 2.525 and 2.578: a tie with the 4-character counting model, not a win. The last position of the window is slightly worse than the middle (2.59 against 2.49); every position is trained equally often, so this lesson does not explain it. The train figure is lower than held-out partly because it is measured on the batches being learned.

WHY CHOOSE THIS?

A model that keeps improving with context and with training.

WHY NOT THIS? (TRADEOFFS)

After 4,000 CPU steps it only matches a lookup table.

5. Characters against BPE tokens

DEFINITION — start here, no prior knowledge assumed

The same model, steps and seed on Day 84's BPE pieces (200 merges, 265 tokens). Loss is converted to bits per character (total bits over the characters the tokens spell), so the two are measured on the same scale and the same held-out text.

REAL-WORLD SCENARIO

Reading syllable by syllable instead of letter by letter: fewer steps per sentence, each step a bigger choice.

MATHEMATICS

$$\textit{bits per character} = \Sigma_{\textit{tokens}} -\log_2 p(\textit{token}) \,/\, (\textit{number of characters the tokens spell})$$

ACTUAL OUTPUT

```
  step  characters     BPE
     0       6.018   4.347
   500       2.988   2.744
  1000       2.866   2.584
  1500       2.731   2.525
  2000       2.688   2.487
  2500       2.627   2.456
  3000       2.623   2.451
  3500       2.594   2.435
  4000       2.564   2.425
with at least 32 tokens of context for every scored token: characters 2.525, BPE 2.399 bits per character
ms per step: characters 317, BPE 333
Actual output from this lesson's run.
```

What it means

BPE is better at every check and ends at 2.425 against 2.564 (windows) and 2.399 against 2.525 (with full context): below the counting model too. Two things changed together: the vocabulary, and how much text a 64-token window covers (about 119 characters instead of 64), so this does not separate the two. The cost per step was similar (333 ms against 317 ms in this run); the token table and output layer grow to 137737 weights.

WHY CHOOSE THIS?

More text per step and per window for the same model.

WHY NOT THIS? (TRADEOFFS)

A bigger output layer, and the comparison mixes vocabulary with context length.

6. What it writes, and the words it makes up

DEFINITION — start here, no prior knowledge assumed

Text drawn from the character model (seed 0), one character at a time from its own probabilities (temperature 1). Then a 5,000-character sample: how many of its words are real, and whether its invented words are made of real pieces, using Day 88's char_ngrams: the share of a word's 3-6-character pieces that occur in training words.

REAL-WORLD SCENARIO

Listening to someone imitate a language they half know: real words, plausible non-words, the right rhythm.

ACTUAL OUTPUT

```
Drawn at temperature 1 (plain sampling, every character from the model's distribution):
Look of you.
KING RICHARD II:
Go; Evenly long, and luck pitience it thy wit gener.
His angive hat talk, but friends thou my he
come been time my patsy the true sourth
Much dauggs, and with he dunk gallieg To mys,
CORINIUS:
Thy drone, and for new-learnior:
My go God! lary, whut our pards my come buts
in a 5,000-character sample: 917 words, 0.710 of them real training words; 249 different invented words of 4+
    letters, e.g. accound, apends, appoor, appress, armitter, arth, backingd, barac
share of each word's character pieces (3-6, Day 88) that occur in training words: invented 0.675, real
    held-out words never seen in training 0.756 (623 words), random letters 0.237
Actual output from this lesson's run.
```

What it means

It has learned the shape of a play: speaker names in capitals followed by a colon, short verse lines, punctuation. 0.710 of its words are real training words; the rest are invented ('accound', 'appress'), and they are made of real pieces: 0.675 of their pieces occur in training words, close to real words it never saw (0.756) and far from random letters (0.237). It does not make sense beyond a few words; Week 14 measures and improves that.

WHY CHOOSE THIS?

Shows what the numbers mean in text.

WHY NOT THIS? (TRADEOFFS)

One sample at one temperature; sampling strategies are Day 93's topic.

7. What the character table learned

DEFINITION — start here, no prior knowledge assumed

Day 89's save and load on the learned 65 × 64 character table, its nearest neighbours by cosine, two simple checks, and Day 87's PCA.

REAL-WORLD SCENARIO

Asking a child who learned to read alone which letters 'feel the same'.

ACTUAL OUTPUT

```
saved to day90_char_vectors.txt and loaded back: same tokens True, largest |difference| 5.0e-06
  'a'    -> 'A' 0.41, 'i' 0.28, 'u' 0.26, 'o' 0.26
  'e'    -> 'E' 0.35, 'U' 0.25, 'u' 0.22, 'i' 0.19
  'T'    -> 't' 0.52, 'X' 0.30, 'W' 0.27, 'v' 0.22
  '?'    -> '!' 0.86, '.' 0.79, ':' 0.54, ';' 0.51
  ' '    -> '-' 0.35, '\n' 0.32, 'X' 0.14, 'C' 0.10
  '\n'   -> ' ' 0.32, 'X' 0.17, 'M' 0.16, '-' 0.14
a lowercase letter's capital is among its 5 nearest characters: 0.885 of 26 letters
a vowel's nearest other lowercase letter is a vowel: 1.000 of 5 (by chance about 4/25 = 0.16)
PCA (Day 87): two directions keep 0.164 of the table's spread
Actual output from this lesson's run.
```

What it means

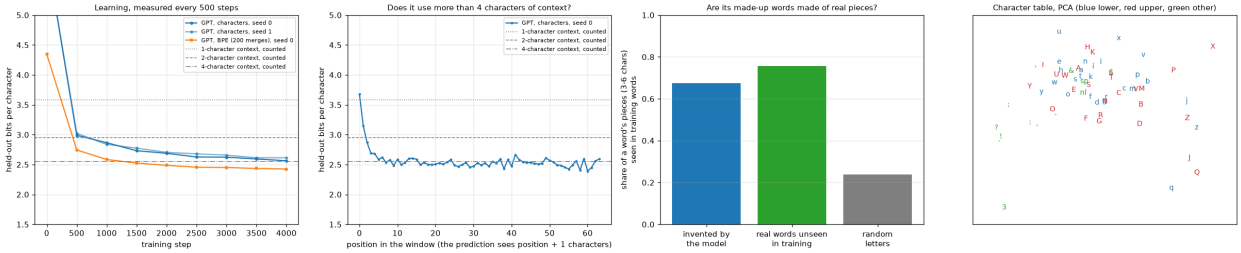
Nobody told the model that 'a' and 'A' are the same letter: for 0.885 of the letters the capital is among the 5 nearest characters. Every vowel's nearest letter is another vowel (1.000, chance about 0.16), and '?' sits next to '!' (0.86) and '.'. Space and newline are neighbours. Two PCA directions keep only 0.164 of the spread, so the right-hand picture shows little of this; the neighbour lists show it better.

WHY CHOOSE THIS?

Learned structure, measurable without labels.

WHY NOT THIS? (TRADEOFFS)

A 2-D picture of a 64-number table hides most of it (Day 87).



Left: held-out bits per character during training, with the counting models. Second: the loss at each position of the window. Third: the share of a word's pieces seen in training words. Right: the character table, PCA.

8. What it cost

ACTUAL OUTPUT

```
characters, seed 0    111937 weights, 317 ms per step, 21.2 minutes for 4000 steps
characters, seed 1    111937 weights, 318 ms per step, 21.2 minutes for 4000 steps
BPE, seed 0           137737 weights, 333 ms per step, 22.2 minutes for 4000 steps
(one CPU thread for the matrix library; only ratios within this run are meaningful)
Actual output from this lesson's run.
```

About 21 minutes per 4,000-step run on one CPU thread, against a counting model that needs no training steps at all, only counting. That is the honest summary of today: the GPT can use long context and keeps improving with training; at this size and budget, it has only just caught up.

Interview preparation — technical questions

Q: What makes a Transformer block 'GPT-style'?

A: A causal mask: position t may attend only to positions 0..t.

Q: Why did Day 78's backward pass work unchanged?

A: The mask only sets some attention weights to 0; the backward pass uses the weights as they are.

Q: How do you compare models with different tokenizers?

A: Bits per character: total bits divided by the characters the tokens spell.

Q: Why measure loss by position in the window?

A: Windows start without context; the curve shows how much context helps and gives a fair score.

Q: How is text generated?

A: Repeatedly: run the model on the last T tokens, draw the next token from its distribution, append it.

Interview preparation — theoretical questions

Q: Why can a 4-gram counter match a Transformer here?

A: Small model, short training, small data: the Transformer's advantage is long context, which it only begins to use.

Q: Why did BPE help?

A: More text per window and per step; the experiment does not separate that from the vocabulary change.

Q: What do the invented words show?

A: The model learned sub-word structure: 0.675 of their pieces are real, against 0.237 for random letters.

Q: Why does the table put 'a' next to 'A'?

A: Likely because they are followed by similar characters, so the model gives them similar vectors (Day 86's idea, at the character level).

Key takeaway and what's next

> **KEY TAKEAWAY**
> Week 13's parts and Week 12's block make a working GPT-style language model, checked entry by entry. It learns the shape of Shakespeare, real sub-word pieces and letter case, and it uses long context; but measured fairly, after 4,000 steps it only ties a four-character counting model, and BPE pieces beat characters at the same size.

- Gradient check: 1218 entries, 1.85e-10; the causal mask holds exactly.
- Held-out bits per character with at least 32 characters of context: 2.525 and 2.578 (two seeds); counting with 4 characters: 2.554.
- By context: 3.68 bits with 1 character, 2.69 with 5, 2.50 with 17.
- BPE (200 merges): 2.399 with full context, the best model today.
- Samples: 0.710 real words; invented words' pieces 0.675 real (random letters 0.237).

Suggested watch order, if starting from zero

- 1. The spelled-out intro to language modeling: building makemore (Andrej Karpathy) — character-level language models, counting first, then neural.
- 2. Transformers, the tech behind LLMs, Deep Learning Chapter 5 (3Blue1Brown) — what a GPT-style Transformer does, visually.

Week 14 preview

- **Mini-GPT and language modeling.** Day 91 takes next-token prediction apart from scratch; Day 92 trains the mini-GPT properly on real text with honest perplexity numbers; Day 93 compares sampling strategies.